

Deep Learning Systems Analysis Report

CNN Image Classification on Fashion-MNIST: A Controlled Comparison of a Plain CNN versus a Batch-Normalised, Dropout-Regularised CNN

Cameron Doelling · Project 4 — Deep Learning Systems · Companion notebook:
deep_learning.ipynb · Code: github.com/doellingcameron92/Deep-Learning-Systems

1. Report Overview

This project addresses 10-class image classification of clothing items: given a 28×28 grayscale product photo, predict its category (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag or Ankle boot). The dataset is Fashion-MNIST, a public benchmark of 70,000 real Zalando product images (Xiao, Rasul, & Vollgraf, 2017). A Convolutional Neural Network (CNN) was implemented in PyTorch (Paszke et al., 2019) as the baseline, and a single controlled change — adding Batch Normalization and Dropout as a regularization strategy while holding every other factor fixed — was evaluated against it. The regularized model reached 92.28% test accuracy versus 91.76% for the baseline, and, more importantly, cut the train–test accuracy gap from 4.4 to 0.4 percentage points.

2. Dataset and Task Description

What the data represents. Fashion-MNIST was released by Zalando Research as a drop-in, harder replacement for the hand-written digit set MNIST. Each sample is a photograph of a real article sold on Zalando's web shop, converted to grayscale, trimmed and down-sampled to 28×28 pixels, and labelled with one of ten product categories (Xiao et al., 2017). The images are therefore neither synthetic nor AI-generated, the dataset is MIT-licensed and widely used in academic work, and it has not been used in any previous capstone project of mine.

Size and structure. The official release contains 60,000 training and 10,000 test images with exactly 6,000 and 1,000 images per class respectively, i.e. it is perfectly balanced. In the notebook the 60,000 training images are split with a fixed random seed into 50,000 for training and 10,000 for validation; the official test set is used exactly once, for the final evaluation of both models. Keeping a validation set separate from the test set is the standard way to select models (here: the epoch with the best validation accuracy) without optimistically biasing the reported test score (Goodfellow, Bengio, & Courville, 2016, ch. 5).

Why it suits a CNN. The inputs are 2-D grids of pixels whose informative structure — edges, sleeves, collars, soles — is local and appears at any position. Convolutional layers share the same small filters across all positions, giving translation-equivariant feature detectors with far fewer parameters than a fully connected network on 784 inputs (LeCun, Bengio, & Hinton, 2015). Fashion-MNIST is also small enough (≈30 MB) to train from scratch on a CPU in minutes, which made it possible to run a fully controlled two-model experiment within this project's constraints.

Preparation. The data is obtained programmatically with the torchvision *FashionMNIST* dataset class (download=True); no manual steps or credentials are required. Pixels are converted from 8-bit integers to floats in [0, 1] and standardised with the training-set mean (0.2860) and standard deviation (0.3530), which the notebook re-computes and verifies. Standardising inputs keeps early-layer

activations well scaled and speeds up gradient-based training (Goodfellow et al., 2016). No data augmentation was applied so that the only difference between the two experiments would be the model's regularization layers. Inspection of representative samples (Figure 1) already suggests the main difficulty: T-shirt/top, Pullover, Coat and Shirt share nearly identical silhouettes and differ in fine texture that is partly destroyed at 28×28 resolution.



Figure 1. Four training examples per class (notebook output).

3. Model Architecture and Design Decisions

The baseline is a deliberately simple, un-regularized CNN so that the experiment isolates the effect of regularization:

Stage	Layers	Output shape
Block 1	Conv2d(1→32, 3×3, pad 1) → ReLU → MaxPool 2×2	32 × 14 × 14
Block 2	Conv2d(32→64, 3×3, pad 1) → ReLU → MaxPool 2×2	64 × 7 × 7
Head	Flatten → Linear(3136→256) → ReLU → Linear(256→10)	10 logits

Layer types and depth. Two convolution–pooling blocks are sufficient for 28×28 inputs: after two 2×2 poolings the feature map is 7×7, so each unit in the head already sees a large fraction of the garment. The 32→64 channel widening follows the common pattern of trading spatial resolution for feature richness as depth increases (Krizhevsky, Sutskever, & Hinton, 2012). Adding further blocks at this resolution gives diminishing returns while increasing CPU training time. The network has 824,458 trainable parameters, most of them in the 3136→256 linear layer.

Activations. ReLU is used after every convolution and the hidden linear layer because it does not saturate for positive inputs, which avoids the vanishing gradients that hamper sigmoid/tanh networks, and it is cheap to compute (Nair & Hinton, 2010; LeCun et al., 2015).

Pooling. Max-pooling halves the resolution twice, giving a degree of invariance to small translations and reducing the number of parameters in the classifier head (Goodfellow et al., 2016, ch. 9).

Output format and loss. The final layer emits ten raw logits, and training minimises *CrossEntropyLoss*, which fuses log-softmax and negative log-likelihood into a single numerically stable operation; this is the standard formulation for single-label multi-class classification (Goodfellow et al., 2016, ch. 6).

Optimiser and training schedule. Both models are trained with Adam at learning rate 1e-3 (Kingma & Ba, 2015), batch size 128, for 12 epochs, with the checkpoint of the best validation epoch retained. Adam's per-parameter adaptive step sizes make it robust without a hand-tuned schedule, which is desirable when the goal is a fair comparison rather than the best possible number. Twelve epochs was chosen after a pilot showed that the un-regularized model begins to over-fit before then, which is the behaviour the experiment is designed to expose. Retaining the best-validation checkpoint is a form of early stopping (Prechelt, 1998).

Observed baseline behaviour. Figure 2 shows the baseline's training loss decreasing monotonically from 0.46 to 0.06 while the validation loss bottoms out at 0.219 in epoch 9 and then climbs to 0.274 by epoch 12; the training accuracy reaches 97.7% while validation accuracy stalls at $\approx 92\%$. This widening gap is the classical signature of over-fitting, in which the network memorises training examples instead of learning transferable features (Goodfellow et al., 2016, ch. 7).

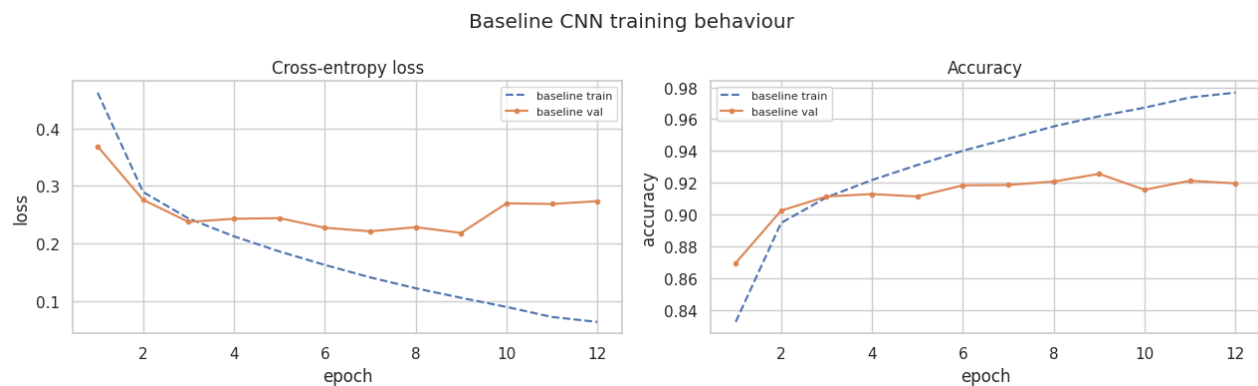


Figure 2. Baseline CNN training/validation loss and accuracy over 12 epochs (notebook output).

4. Experimental Comparison

What changed — exactly one aspect: the regularization strategy. The experimental model keeps the baseline's exact topology (same convolutions, pooling, linear layers and widths), loss, optimiser, learning rate, batch size, number of epochs, random seed and data split. It adds only (i) *BatchNorm2d* after each convolution and *BatchNorm1d* after the hidden linear layer, and (ii) *Dropout* with $p = 0.25$ after each pooling stage and $p = 0.5$ before the output layer. This changes the parameter count by just 704 affine BatchNorm parameters (+0.09%), so the comparison is not confounded by model capacity.

	Baseline CNN	Regularized CNN
Conv / pool / linear topology	2 blocks (32, 64) + FC-256 + FC-10	identical
Batch Normalization	none	after each conv and after FC-256
Dropout	none	0.25 after each pool, 0.5 before output
Loss, optimiser, lr, batch, epochs, seed, split	CE, Adam 1e-3, 128, 12, 42, 50k/10k	identical

Trainable parameters	824,458	825,162 (+704)
----------------------	---------	----------------

Why this is a meaningful comparison. The baseline's diagnosis is over-fitting, and Dropout and Batch Normalization are the two most widely used architectural remedies. Dropout randomly zeroes units during training, which prevents co-adaptation and approximates averaging an exponential number of thinned sub-networks (Srivastava, Hinton, Krizhevsky, Sutskever, & Salakhutdinov, 2014). Batch Normalization standardises each layer's inputs per mini-batch, which smooths optimisation and adds mild stochastic noise that also acts as a regulariser (Ioffe & Szegedy, 2015). Both target the same failure mode and are routinely introduced together, so adding them constitutes a single conceptual intervention. Because everything else is fixed, any difference in the curves and metrics can be attributed to that intervention. The expected trade-offs were a lower *training* accuracy (Dropout handicaps the network at train time), more compute per epoch, and — if the hypothesis is right — a lower validation/test loss and a smaller generalisation gap.

5. Results and Interpretation

Both selected checkpoints were evaluated once on the untouched 10,000-image test set. Accuracy is an appropriate headline metric because the classes are perfectly balanced; macro-F1 and per-class precision/recall are reported as well so that a weak class cannot hide behind strong ones (Sokolova & Lapalme, 2009).

Metric	Baseline CNN	Regularized CNN	Δ (reg – base)
best epoch (val)	9	11	+2
train acc @best	0.9618	0.9267	-0.0351
val acc @best	0.9256	0.9269	+0.0013
min val loss	0.2186	0.1982	-0.0204
final-epoch val loss	0.2736	0.2017	-0.0719
test loss	0.2544	0.2179	-0.0364
test acc	0.9176	0.9228	+0.0052
test macro-F1	0.9169	0.9221	+0.0053
generalisation gap (train-test acc)	0.0442	0.0039	-0.0403
mean epoch time (s)	7.5	11.2	+3.7

Table 1. Summary metrics for the two models (notebook output). Loss values are cross-entropy.

Training behaviour: baseline vs. regularized CNN (same optimizer, lr, batch size, epochs, seed)

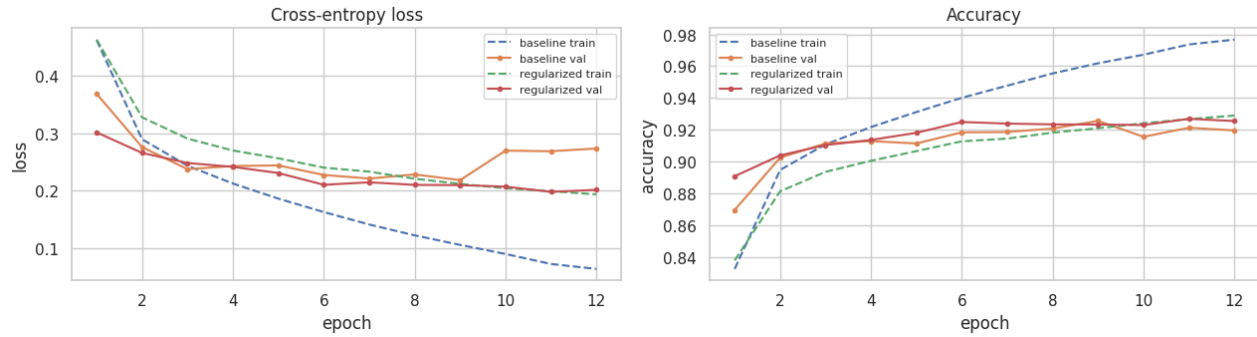


Figure 3. Training behaviour of both models under identical settings (notebook output).

Generalisation is the main effect, not raw accuracy. The regularized model's test accuracy is only 0.52 percentage points higher (92.28% vs 91.76%, i.e. about 52 more correct images out of 10,000). What changed decisively is the *shape* of training: Figure 3 shows the regularized model's validation loss still decreasing at epoch 12 (0.198 minimum, 0.202 final) while the baseline's has turned upward (0.219 $\rightarrow$ 0.274). Test loss fell from 0.254 to 0.218, and the train–test accuracy gap collapsed from 4.4 to 0.4 percentage points. In other words, the regularized network fits the training data *less* well (92.7% vs 96.2% training accuracy at the selected epoch) but the unseen data better — precisely the trade Dropout is designed to make (Srivastava et al., 2014).

Where the gains come from. Per-class F1 improved for every one of the four upper-body garments — T-shirt/top (+0.012), Pullover (+0.014), Coat (+0.015), Shirt (+0.007) — and for Dress (+0.013), while the already near-perfect classes (Trouser, Sandal, Sneaker, Bag, Ankle boot; $F1 \geq 0.96$) moved by at most ± 0.004 . The improvement is therefore concentrated exactly where a memorising model is most fragile: on classes separated by subtle texture cues rather than gross shape. Table 2 shows the largest confusion cells.

True $\rightarrow$ Predicted	Baseline	Regularized
Shirt $\rightarrow$ T-shirt/top	95	101
Shirt $\rightarrow$ Coat	90	85
T-shirt/top $\rightarrow$ Shirt	88	80
Shirt $\rightarrow$ Pullover	83	69
Coat $\rightarrow$ Pullover	69	46

Table 2. Most frequent test-set confusions (counts out of 1,000 images per class; notebook output).

A concrete example of model behaviour: the Shirt class. Shirt is by far the hardest class for both models (recall 0.70 vs 0.71; $F1$ 0.750 vs 0.758) and it accounts for the four largest confusion cells. Regularization repaired many Shirt $\rightarrow$ Pullover (83 $\rightarrow$ 69) and Coat $\rightarrow$ Pullover (69 $\rightarrow$ 46) errors, yet the single largest cell, Shirt $\rightarrow$ T-shirt/top, actually grew from 95 to 101. That is, the regularized model shifted where it fails rather than eliminating failures: it became better at long-sleeve versus short-sleeve distinctions but no better at separating a collared shirt from a T-shirt when both are short-sleeved. The notebook's 'both wrong' panel (564 test images misclassified by both models) is dominated by exactly these cases, many of which are ambiguous to a human at 28×28 resolution.

The models are not strictly nested. 260 test images were fixed by regularization but 208 were broken by it. The net gain of 52 images matches the accuracy delta, but the churn shows that a 0.5-point improvement from a single seed should be read as 'at least not worse, and probably slightly better', not as a decisive ranking. The loss and generalisation-gap results are far larger relative to their scale and are the more trustworthy evidence.

Calibration. The baseline is more over-confident when it is wrong: its mean maximum-softmax probability on misclassified test images is 0.749 versus 0.710 for the regularized model, and 26.1% of its errors carry confidence above 0.9 compared with 19.6%. Over-confident errors are a known property of modern high-capacity networks trained to low training loss (Guo, Pleiss, Sun, & Weinberger, 2017); they matter in practice because a deployment that abstains below a confidence threshold would catch fewer of the baseline's mistakes.

Cost. The regularized model needed 11.2 s per epoch versus 7.5 s (+50%) on the same 8-core CPU, because BatchNorm adds extra passes over every activation. For a 12-epoch run this is under a minute, but for larger models it is a real trade-off against the generalisation benefit.

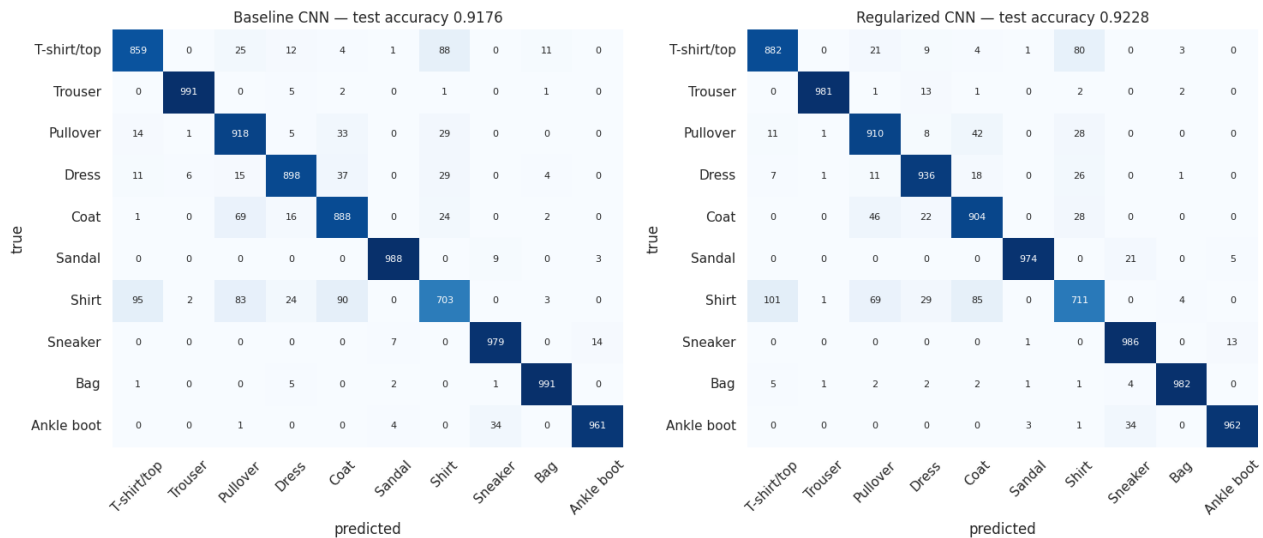


Figure 4. Test-set confusion matrices for both models (notebook output).

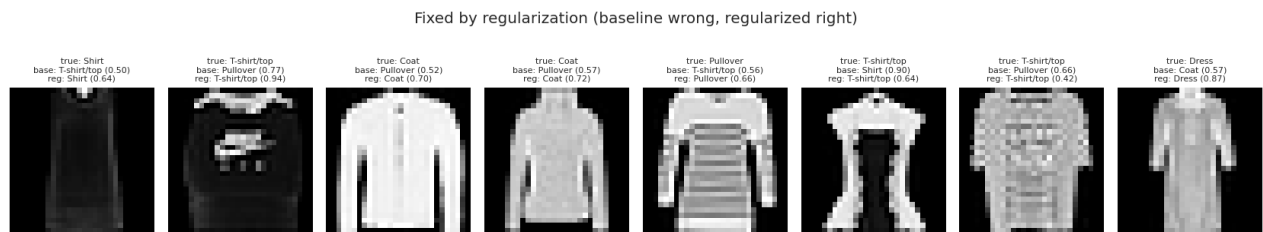


Figure 5. Test images the baseline misclassified and the regularized model classified correctly, with each model's top-class confidence (notebook output).

6. Limitations and Risks

- **Single seed, single split.** Every number above comes from one training run with seed 42. The accuracy difference (0.52 pp, 52 images) is within the range that seed-to-seed variation can produce for CNNs of this size; without repeated runs and confidence intervals the accuracy

ranking is suggestive, not established. The loss and generalisation-gap effects are large enough that this concern applies much less to them.

- **Two changes bundled as one intervention.** BatchNorm and Dropout were introduced together as one 'regularization strategy'. The experiment shows that the bundle helps, but cannot attribute the gain between them; an ablation with each alone would be required.
- **Dataset limitations.** Fashion-MNIST images are 28×28 grayscale, centred, on a uniform background and drawn from a single retailer's catalogue. Texture, colour and context that distinguish a Shirt from a T-shirt in reality are largely absent, which caps achievable accuracy and produces the Shirt confusions documented above. A model trained here would not transfer to user-uploaded photos with clutter, varying scale, or colour.
- **Architecture and training limitations.** The CNN is intentionally shallow, uses no data augmentation, no learning-rate schedule and no weight decay. Twelve epochs and a fixed learning rate were chosen for a fair comparison, not for peak performance; published results on Fashion-MNIST with deeper, augmented networks exceed 94–95% (Xiao et al., 2017).
- **Evaluation limitations.** Model selection used validation accuracy only; the test set is the official split and was used once, but it comes from the same distribution as training, so the reported accuracy is an in-distribution estimate and says nothing about robustness to distribution shift. Calibration was assessed only with mean confidence and a 0.9 threshold rather than a full reliability diagram or expected calibration error (Guo et al., 2017).

7. Ethical and Responsible Use

Representation bias. Fashion-MNIST is drawn entirely from one European retailer's catalogue, so its ten categories and the visual style of each reflect that retailer's inventory and photography. Garments common in other regions or cultures (e.g. saris, kurtas, abayas, kimonos) have no class and would be forced into the nearest Western category. A catalogue-tagging system built on such a model would systematically mislabel or hide products from sellers whose goods fall outside the training distribution, an example of representation harm arising from a non-representative dataset (Buolamwini & Gebru, 2018).

Over-confident automation. The results show that a quarter of the baseline's errors carry confidence above 0.9. If such a model were used to automate product listing or returns processing without a human in the loop, its mistakes would be both frequent for the Shirt class (≈30% error rate) and hard to detect by confidence thresholding. Responsible deployment would require per-class error reporting, calibration checks, and human review for low-margin classes; documenting these limits in a model card is the recommended practice (Mitchell et al., 2019).

Misuse risk. The model itself is low-risk — it classifies clothing categories, not people — but the same pipeline could be retrained on images of people's clothing to infer attributes such as religious dress or socioeconomic status for profiling or targeting. The techniques demonstrated here are domain-agnostic, so the responsibility lies in the choice of training data and application, which should be documented and constrained.

8. Future Improvements

- **Repeat with multiple seeds** (e.g. 5 runs each) and report mean $\pm$ standard deviation and a paired test, to turn the 0.5-point accuracy difference into a statistically supported claim.
- **Ablate BatchNorm and Dropout separately**, and sweep Dropout probability (0.1–0.5), to attribute the improvement and find the best regularization strength.
- **Add data augmentation** (random crops, small rotations, horizontal flips) as the next single controlled change; it attacks over-fitting from the data side rather than the model side and typically yields larger gains on Fashion-MNIST than architectural regularization alone (Goodfellow et al., 2016, ch. 7).
- **Target the Shirt / T-shirt / Pullover confusion** with a deeper residual-style network or higher input resolution, and inspect learned filters or Grad-CAM maps to understand which pixels drive the confusions.
- **Improve and measure calibration** with temperature scaling and a reliability diagram (Guo et al., 2017), and add a confidence-based abstention path for deployment.
- **Test under distribution shift** by evaluating on a small set of real, colour, un-centred product photos to quantify how far in-distribution accuracy overstates real-world performance, and publish a model card.

References

Citation style: APA 7th edition.

- Buolamwini, J., & Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. *Proceedings of the 1st Conference on Fairness, Accountability and Transparency, PMLR 81*, 77–91.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org>
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning, PMLR 70*, 1321–1330.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning, PMLR 37*, 448–456.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1412.6980>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, 220–229. <https://doi.org/10.1145/3287560.3287596>
- Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 807–814.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang,

- L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 8024–8035.
- Prechelt, L. (1998). Early stopping — but when? In G. B. Orr & K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* (Lecture Notes in Computer Science, Vol. 1524, pp. 55–69). Springer.
https://doi.org/10.1007/3-540-49430-8_3
- Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437. <https://doi.org/10.1016/j.ipm.2009.03.002>
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 1929–1958.
- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*. <https://arxiv.org/abs/1708.07747>